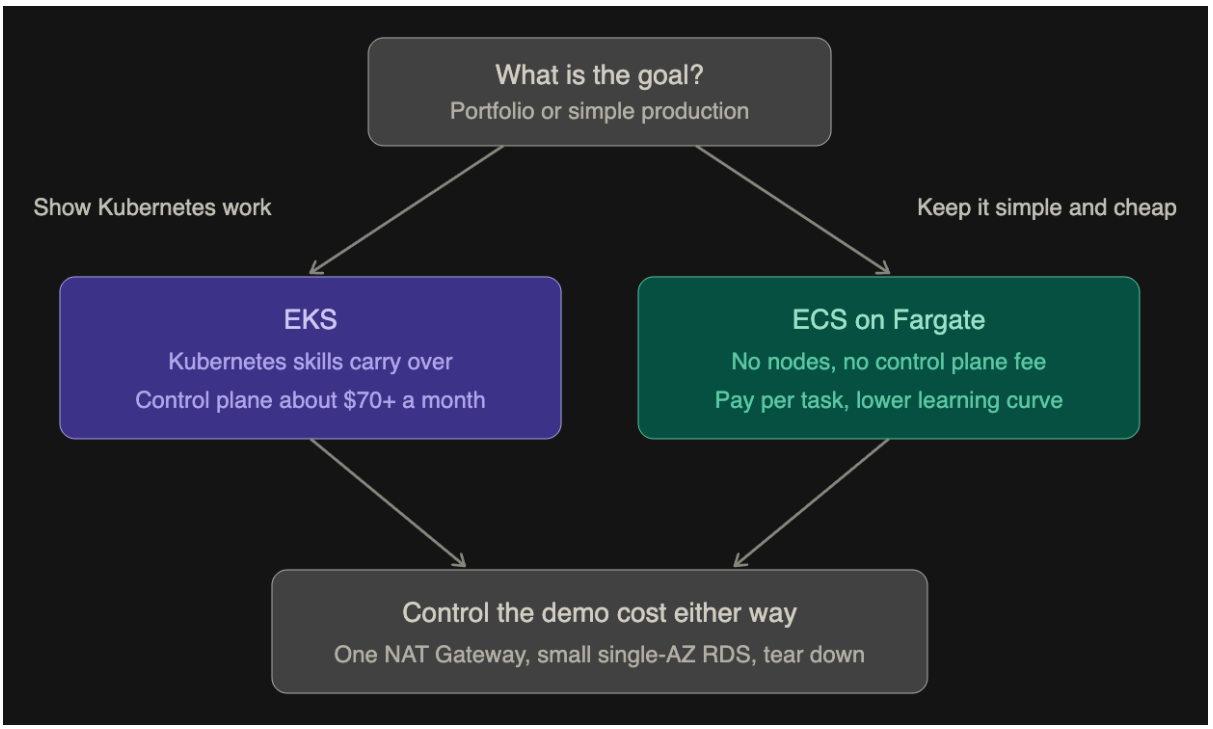
EKS or ECS on Fargate: Which Container Platform Fits Your Project?

The choice comes down to one question: is the goal to show Kubernetes skills, or to run something simple and cheap? Here is the decision as a diagram, then the details.



	EKS	ECS on Fargate
What it is	AWS's managed Kubernetes service	AWS's own container orchestrator, with Fargate running the containers so you manage no servers
Fits because	If you already know Kubernetes, and the skill is portable to any cloud or employer	It is simpler, with no nodes to manage
Learning curve	Higher (pods, deployments, services, ingress, cluster setup)	Lower (task definitions, services, a load balancer)
Cost model	The control plane costs roughly $70+ per month before you add any nodes	No control plane fee, and you pay per task for the CPU and memory you use

Work if you already have Kubernetes manifests	Little, since your manifests largely carry over	A rewrite into task definitions, ALB target groups and service definitions
Portability	High, because Kubernetes runs on any cloud	Lower, because ECS is AWS-specific

What each option means in practice

EKS. You get a managed Kubernetes control plane, and you still run the workloads on nodes (EC2 or Fargate). You carry over the Kubernetes concepts you already know, and that is the strongest argument for it. The cost is the control plane fee plus the worker capacity, load balancers and networking.

ECS on Fargate. You describe each container as a task definition, put it behind a service and a load balancer, and AWS runs it without servers for you to patch or scale. It is the quickest way to a working production setup, and it has no per-cluster fee.

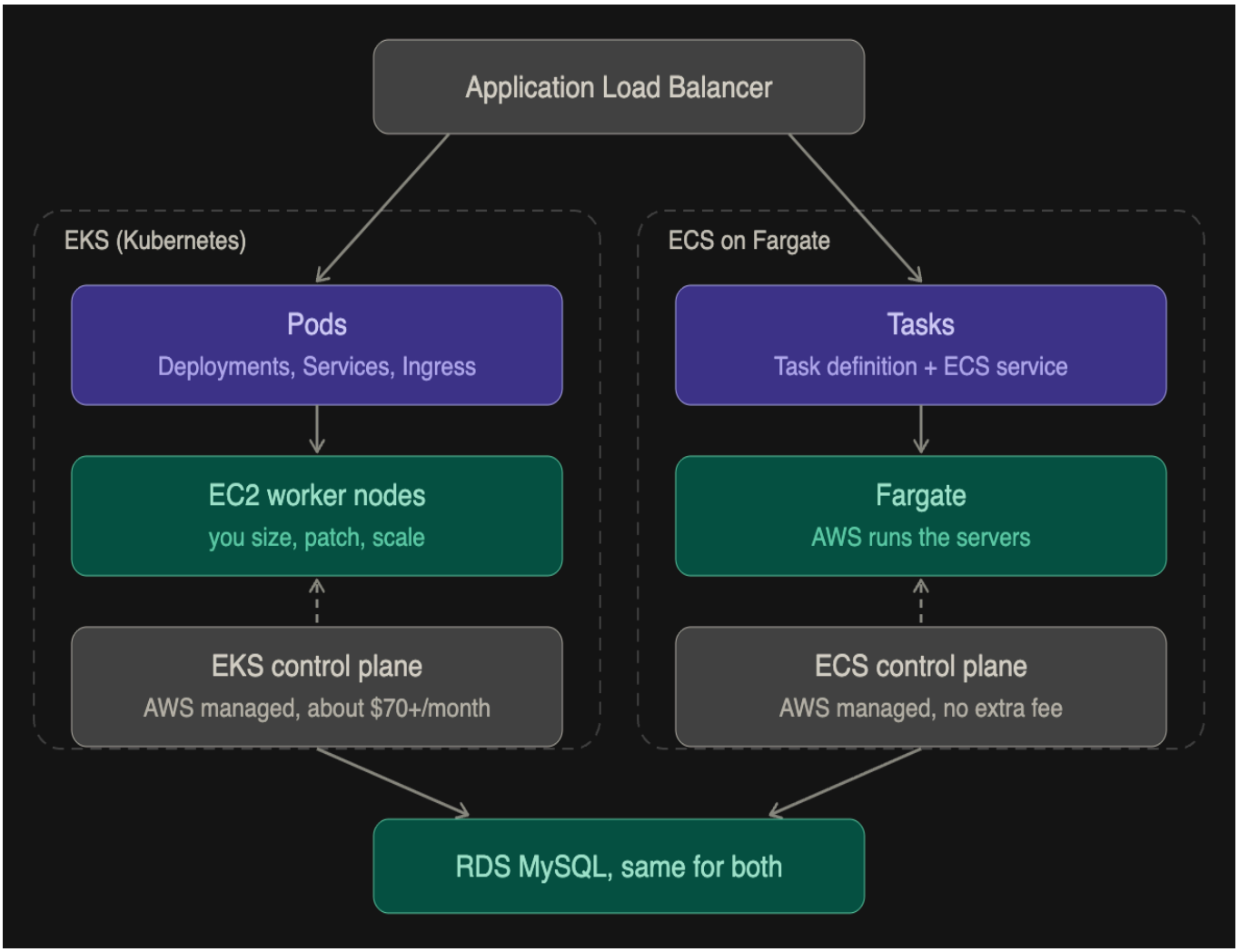
How to decide

- **Choose EKS** if the project is a portfolio piece and you want to show Kubernetes work, or if you already have Kubernetes manifests and want to reuse them.
- **Choose ECS on Fargate** if you want the cheapest, simplest production setup and don't need Kubernetes skills to be the point.

Costs to watch (these apply to both)

The control plane is not the only expense. EKS plus NAT Gateways plus Multi-AZ RDS adds up quickly. NAT Gateways charge by the hour and by data processed, a Multi-AZ database roughly doubles the database cost, and load balancers add a charge each. Check current AWS pricing before committing, because prices change.

Here is a side-by-side picture of the two options. Both use the same load balancer and the same RDS database, so the only difference is how the containers are run.



1. **Workload:** the thing that runs your container. On EKS this is a pod, created by a Deployment. On ECS it is a task, created by an ECS service from a task definition.
2. **Compute:** the machine under it. On EKS you run EC2 worker nodes, so you choose the instance type, patch the OS and scale the node group. On Fargate there are no servers to see. You only say how much CPU and memory each task needs.
3. **Control plane:** the scheduler that decides what runs where. AWS manages it in both cases. EKS charges a flat hourly fee for it, and ECS does not.

EKS to ECS on Fargate: Concept Mapping

How each Kubernetes concept maps to ECS on Fargate, and what it means in the project.

Kubernetes (EKS)	ECS on Fargate	Your project
Pod	Task	One Spring Boot container
Deployment	ECS service	Keeps 2 backends running, does rolling updates
backend-deployment.yaml	Task definition (JSON)	Image, CPU, memory, env vars, secrets
Service plus Ingress	ALB listener rules plus target group	api. host goes to backend, app. host goes to frontend
Secret	Secrets Manager reference in the task definition	SPRING_DATASOURCE_* values
Liveness and readiness probes	ALB health check, container health check	/actuator/health
startupProbe	healthCheckGracePeriodSeconds	Covers your slow Spring Boot start
HPA	ECS service auto scaling	Scale on CPU
Node	Not visible	Not needed

What you would need to change for ECS

- **Convert** `backend-deployment.yaml` and `frontend-deployment.yaml` into **two task** definitions.
- **Replace** the Ingress with one ALB, with a listener rule per host and one target group per service.
- Set the target group health check path to `/actuator/health`. This has to be allowed in Spring Security's `permitAll()`, or the tasks will be marked unhealthy and replaced in a loop, as happened with the 403 on your probes.
- Set a health check grace period of 120 to 180 seconds, since your backend takes about a minute to start.
- Give each Spring Boot task at least 1 vCPU and 2 GB of memory. The very small sizes will slow startup the way your 500m CPU limit did.
- Keep Flyway, RDS, the Docker image and the Spring config **unchanged**.

With EKS, almost all of your existing YAML carries over. Only the ingress annotations, the storage class and the secret source change.

Cost, in plain terms

- **EKS:** a flat control-plane fee (roughly $70+ per month) plus the EC2 nodes, whether or not they are busy. Nodes can be shared by many pods, so it gets cheaper per app as you add more apps.

- **ECS on Fargate:** no control-plane fee, and you pay per task for the vCPU and memory it uses while running. For a small app with 2 or 3 tasks this is usually cheaper.
- **Shared costs in both:** the ALB, NAT Gateway, RDS and data transfer. NAT Gateways and Multi-AZ RDS are often the biggest items, so check the current AWS pricing pages before you commit.

Which to choose:

If your goal is...	Choose
Show Kubernetes skills in a portfolio	**EKS**
Cheapest and simplest production setup	**ECS on Fargate**
Many services, or a team that already knows Kubernetes	**EKS**
One small app, no dedicated ops person	**ECS on Fargate**
Possible move to another cloud later	**EKS (portable YAML)**